



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по практической работе №2

по дисциплине «Разработка мобильных приложений»

Выполнили:

Студенты группы ИКБО-20-23

Кузнецов Л.А.

Проверил:

Старший преподаватель кафедры
МОСИТ

Шешуков Л.С.

Москва 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	3
1.1 Активность.....	3
1.2 Логирование.....	5
1.3 Взаимодействие с элементами пользовательского интерфейса.....	8
1.4 Переход между экранами	12
1.5 Передача данных между Activity	16
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	19
2.1 Отслеживание жизненного цикла активности при помощи логирования	19
2.1.1 Реализация логирования в методах жизненного цикла	19
2.1.2 Тестирование отслеживания жизненного цикла активности	23
2.2 Переход на другую активность.....	26
2.2.1 Создание разметки первой активности.....	26
2.2.2 Создание второй активности и её разметки	29
2.2.3 Реализация перехода на другую активность	31
2.2.4 Реализация передачи данных из первой активности во вторую	33
ЗАКЛЮЧЕНИЕ	37

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Активность

Активность в Android — это основной компонент приложения, который предоставляет интерфейс для взаимодействия пользователя с приложением. Она действует как одно «окно» в пользовательском интерфейсе, через которое пользователь может взаимодействовать с приложением, например, вводить данные, просматривать информацию или выполнять другие действия.

Каждая активность обычно заполняется различными элементами управления, такими как кнопки, текстовые поля, изображения и другие виджеты, которые обеспечивают функциональность приложения. В Android каждая активность имеет свой жизненный цикл, который управляется операционной системой (рисунок 1.1.1).

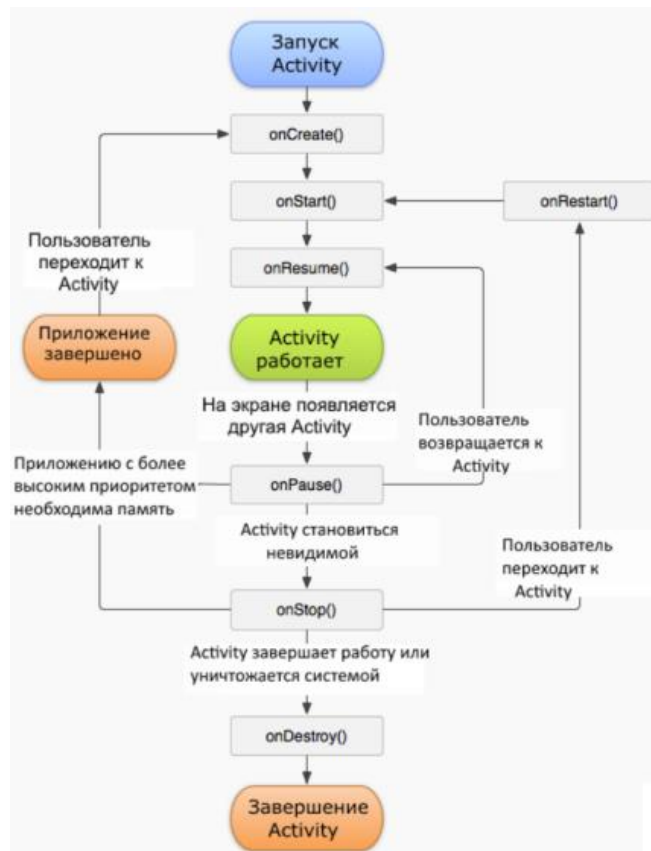


Рисунок 1.1.1 — Жизненный цикл активности

Первым методом в жизненном цикле активности является "onCreate()". Этот метод вызывается при первоначальном создании активности. Здесь разработчики обычно размещают код для инициализации, который нужно выполнить один раз: настройка пользовательского интерфейса, инициализация данных и состояния активности. После "onCreate()", активность переходит в состояние "onStart()".

Метод "onStart()" вызывается, когда активность становится видимой для пользователя. В этом методе можно производить подготовительные действия, чтобы активность была готова к взаимодействию с пользователем. После "onStart()", следующим шагом в жизненном цикле является "onResume()".

"onResume()" активируется, когда активность готова начать взаимодействие с пользователем. На этом этапе активность находится на переднем плане и может принимать пользовательский ввод. Это идеальное место для запуска анимаций или выполнения вычислений, необходимых для интерфейса.

Когда активность переходит в фоновый режим, вызывается метод "onPause()". Этот метод используется для приостановки операций, которые не должны продолжаться, пока активность не находится на переднем плане. Это включает в себя остановку анимаций, приостановку выполнения сложных вычислений или освобождение системных ресурсов.

Если активность становится полностью невидимой для пользователя, система вызывает "onStop()". В этом методе следует останавливать более сложные операции, которые ненужны или неэффективны, когда активность не видна. Если активность снова станет видимой, вызывается "onRestart()", что означает возврат к "onStart()".

Метод "onRestart()" используется для перезапуска операций, которые были остановлены или приостановлены в "onStop()". Он подготавливает активность к повторному запуску и восстановлению её работы.

Наконец, "onDestroy()" вызывается при уничтожении активности. Этот метод используется для окончательной очистки ресурсов и сохранения

данных, если это необходимо. Это последний вызов в жизненном цикле активности, после которого активность полностью уничтожается системой.

Для того чтобы лучше разобраться в работе методов жизненного цикла применим логирование.

1.2 Логирование

Логирование — это ключевой аспект разработки приложений, который позволяет разработчикам отслеживать работу приложения и диагностировать возможные проблемы. В Android предусмотрены два основных способа логирования: использование класса Log и отображение всплывающих сообщений с помощью Toast.

Класс Log в Android используется для записи отладочной информации.

Log предоставляет различные уровни логирования, такие как DEBUG (отладка), ERROR (ошибка), INFO (информация), VERBOSE (подробности) и WARN (предупреждение) и запоминается по первым буквам, позволяя разработчикам классифицировать важность сообщений. Приведём пример записи лога информационного сообщения с пользовательским тегом "MyAppTag" для идентификации (рисунок 1.2.1).

```
Log.i("MyAppTag", "Информационное сообщение");
```

Рисунок 1.2.1 — Пример записи лога

Пользовательский тег также можно определить один для всего приложения через переменную (рисунок 1.2.2).

```

private static final String TAG = "HelloWorld";
@Override
protected void onStart(){
    super.onStart();
    Log.e(TAG, msg: "error in onStart");
    Log.w(TAG, msg: "warning in onStart");
    Log.i( tag: "IKB0-07-22", msg: "info in onStart");
    Log.d( tag: "IKB0-07-22", msg: "debug in onStart");
    Log.v( tag: "IKB0-07-22", msg: "verbose in onStart");
}

```

Рисунок 1.2.2 — Определение пользовательского тега для всего приложения

Сообщения, залогированные с помощью Log, выводятся в панели Logcat, инструменте для просмотра логов, доступном в среде разработки Android Studio (рисунок 1.2.3).

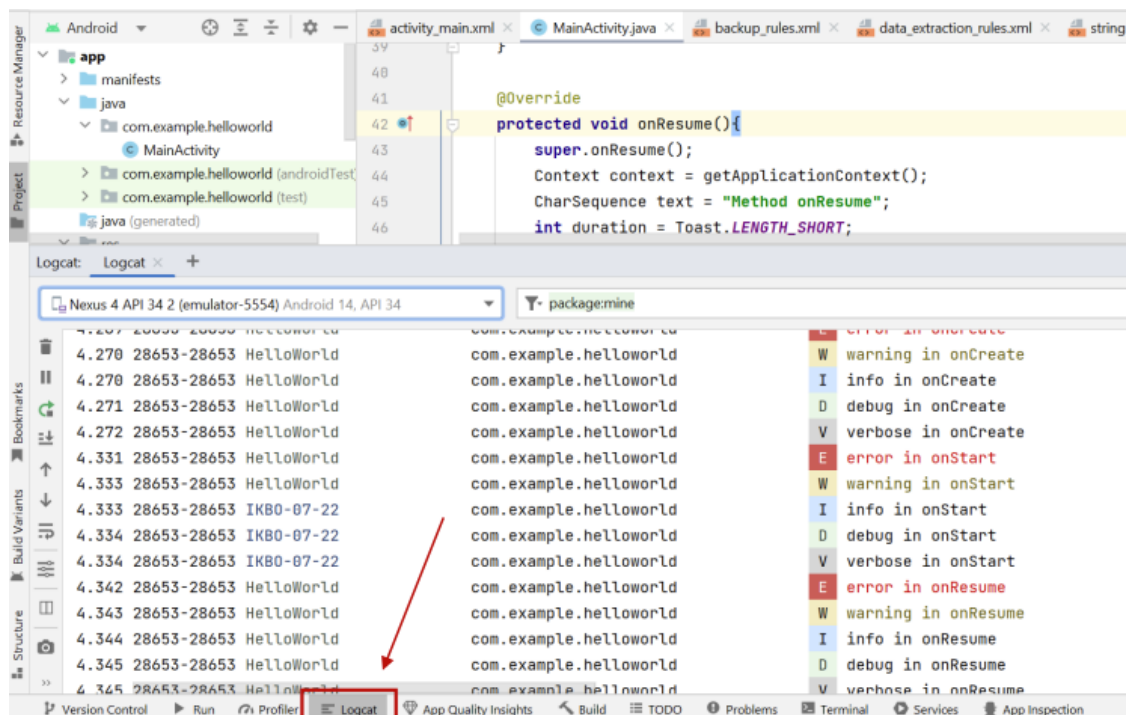


Рисунок 1.2.3 — Панель Logcat

С другой стороны, Toast в Android предназначен для отображения коротких всплывающих сообщений пользователю. Эти сообщения автоматически исчезают после небольшого промежутка времени и не требуют активного взаимодействия пользователя. Toast является эффективным способом для отображения простых уведомлений, таких как подтверждение выполнения какого-либо действия. Приведём пример использования Toast, где "Текст сообщения" — это содержание уведомления, а

Toast.LENGTH_SHORT указывает на короткую продолжительность его отображения (рисунки 1.2.4).

```
Toast.makeText(getApplicationContext(), "Текст сообщения",  
Toast.LENGTH_SHORT).show();
```

Рисунок 1.2.4 — Пример создания сообщения с помощью Toast

Приведём ещё один пример создания и отображения Toast (рисунки 1.2.5–1.2.6).

```
@Override  
protected void onResume(){  
    super.onResume();  
    Toast.makeText(getApplicationContext(), text: "ИКБО-07-22", Toast.LENGTH_SHORT).show();  
}
```

Рисунок 1.2.5 — Создание сообщения с помощью Toast

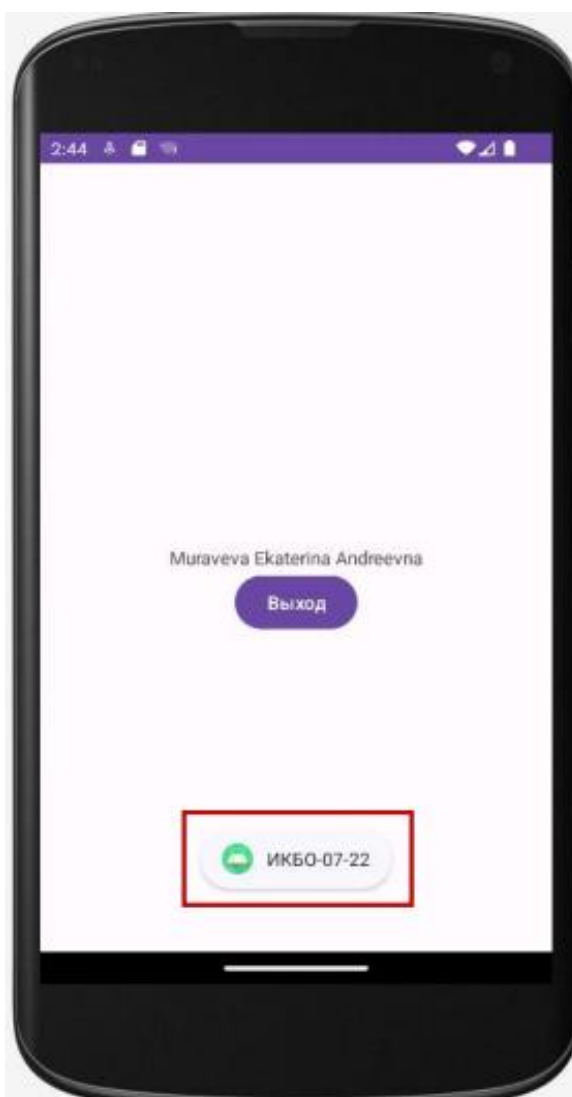


Рисунок 1.2.6 — Отображение созданного Toast-сообщения

Главное отличие между Log и Toast заключается в их целях и способах использования. Log ориентирован на логирование информации для разработчиков и отладку приложения, при этом сообщения Log видны только в Logcat и не отображаются в пользовательском интерфейсе приложения. В отличие от этого, Toast предназначен для взаимодействия с пользователем, показывая короткие информационные сообщения непосредственно в интерфейсе приложения. Также важно отметить, что сообщения Log могут сохраняться в Logcat в течение длительного времени, в то время как Toast отображается только на ограниченный период и затем автоматически исчезает.

1.3 Взаимодействие с элементами пользовательского интерфейса

Взаимодействие с элементами пользовательского интерфейса в приложениях для Android, осуществляется посредством специализированных методов и механизмов. Задать метод обработчик события можно двумя способами:

- декларативно, при помощи атрибута кнопки `onClick`,
- программно, в коде вашего приложения используя метод. устанавливающий обработчик событий для кнопки `setOnClickListener()`.

При программном варианте задания обработчика события два основных метода, которые играют ключевую роль в этом процессе, это `"findViewById"` и `"setOnClickListener"`.

`findViewById` — это метод, используемый для получения ссылки на виджет по его идентификатору. Каждый элемент интерфейса в XML-разметке имеет уникальный ID, который используется в коде для обращения к этому элементу (рисунок 1.3.1).


```

<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/my_button"
    android:text="Следующая страница"
    android:onClick="onNextActivity"
    app:layout_constraintTop_toBottomOf="@id/textView"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"/>

```

Рисунок 1.3.1 — Пример id элемента

Например, если в XML есть кнопка с идентификатором `@+id/my_button`, то для получения ссылки на эту кнопку в Java используется следующий код, представленный на рисунке 1.3.2.

```
Button myButton = findViewById(R.id.my_button);
```

Рисунок 1.3.2 — Получение ссылки на кнопку по её id

После получения ссылки на элемент интерфейса можно взаимодействовать с ним, изменяя его свойства, вызывая методы и так далее.

В свою очередь, `"setOnClickListener"` — это метод, который используется для установки обработчика нажатий на виджет, например, на кнопку. После того как кнопка найдена с помощью `"findViewById"`, можно установить для неё слушатель нажатий. Этот слушатель определяет действия, которые будут выполняться при нажатии на кнопку (рисунок 1.3.3).

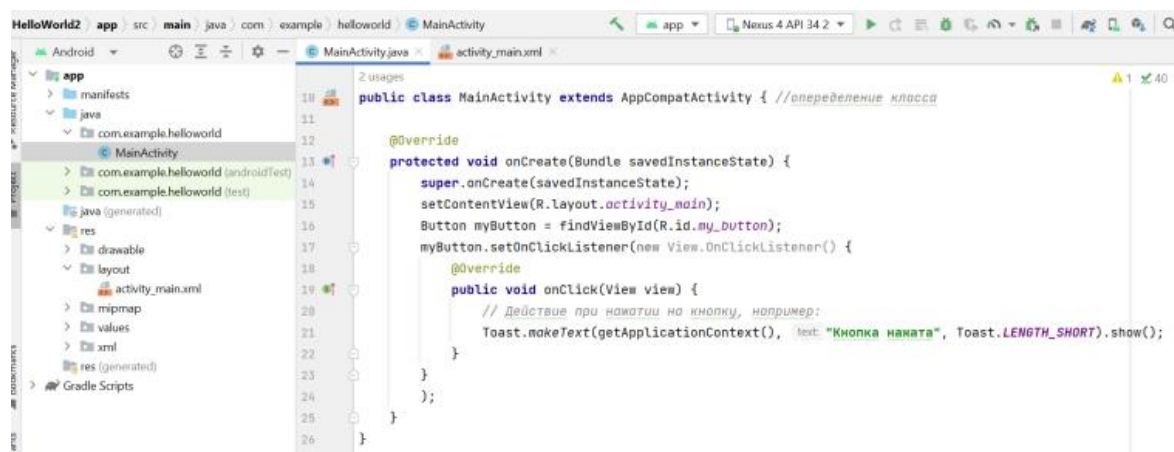


Рисунок 1.3.3 — Пример создания обработчика событий

В этом примере при нажатии на кнопку на экране появится короткое сообщение Toast с текстом «Кнопка нажата» (рисунок 1.3.4).



Рисунок 1.3.4 — Демонстрация работы обработчика событий

Таким образом, "setOnClickListener" позволяет определить интерактивное поведение элементов пользовательского интерфейса, реагирующее на действия пользователя. Так же одним из вариантов того, что можно разместить в слушателе нажатий, является переход между разными экранами в приложении.

Использование атрибута onClick – это относительно новый способ для взаимодействия с элементами пользовательского интерфейса, специально разработанный для Android (рисунок 1.3.5).

```
android:onClick="onNextActivity"
```

Рисунок 1.3.5 — Назначение обработчика клика в XML

Далее нужно прописать в классе активности придуманное вами имя метода, который будет обрабатывать нажатие. Метод должен быть открытым (public) и с одним параметром, использующим объект View (рисунок 1.3.6).

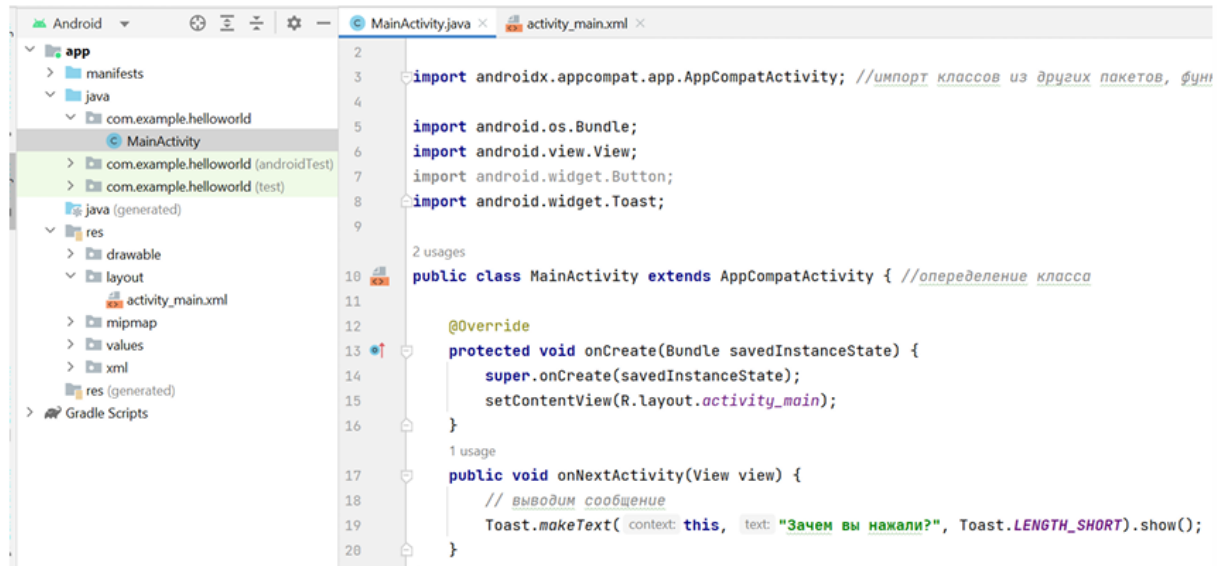


Рисунок 1.3.6 — Описание обработчика события onClick в Java

Когда пользователь нажимает на кнопку, то вызывается метод onNextActivity(), который в свою очередь генерирует всплывающее сообщение (рисунок 1.3.7).



Рисунок 1.3.7 — Результат работы обработчика событий

Обратите внимание, что при подобном подходе вам не придётся даже объявлять кнопку через конструкцию `Button myButton = findViewById(R.id.my_button)`, так как Android сама поймёт, что к чему. Данный способ применим не только к кнопке, но и к другим элементам и позволяет сократить количество строк кода.

1.4 Переход между экранами

В Android-приложениях часто используется несколько активностей, каждая из которых представляет собой отдельный экран или пользовательский интерфейс. Для создания новой активности в Android Studio необходимо выполнить несколько шагов.

Во-первых, в проекте нужно создать новый класс активности. Это можно сделать, нажав на модуль "app", затем в верхней панели выбрав "File" → "New" → "Activity" и затем выбрав тип активности, например "Empty Views

Activity" либо нажать правой кнопкой мыши на "java" → "New" → "Activity" (рисунки 1.4.1–1.4.2).

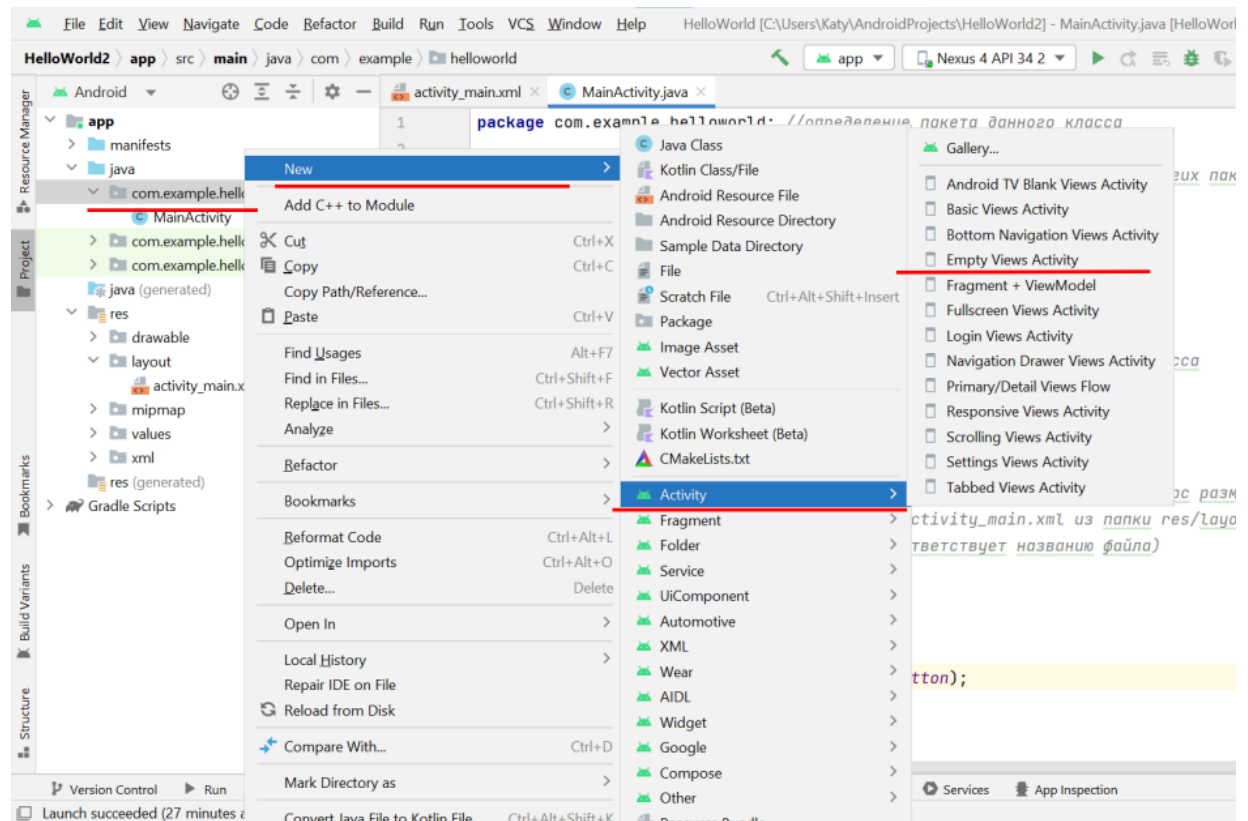


Рисунок 1.4.1 — Создание новой activity

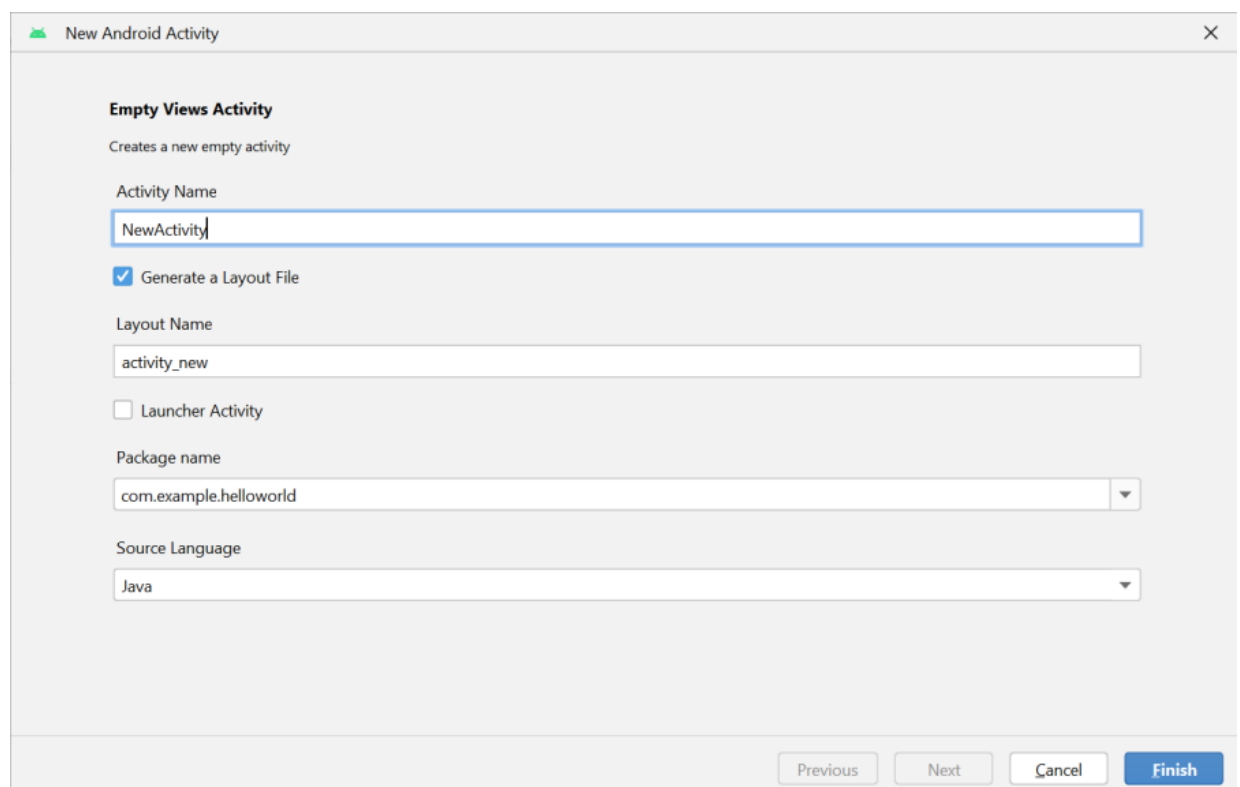


Рисунок 1.4.2 — Первичная настройка при создании новой activity

Android Studio сгенерирует необходимые файлы и ресурсы, такие как файл разметки XML для интерфейса и Java класс для логики активности (рисунок 1.4.3).

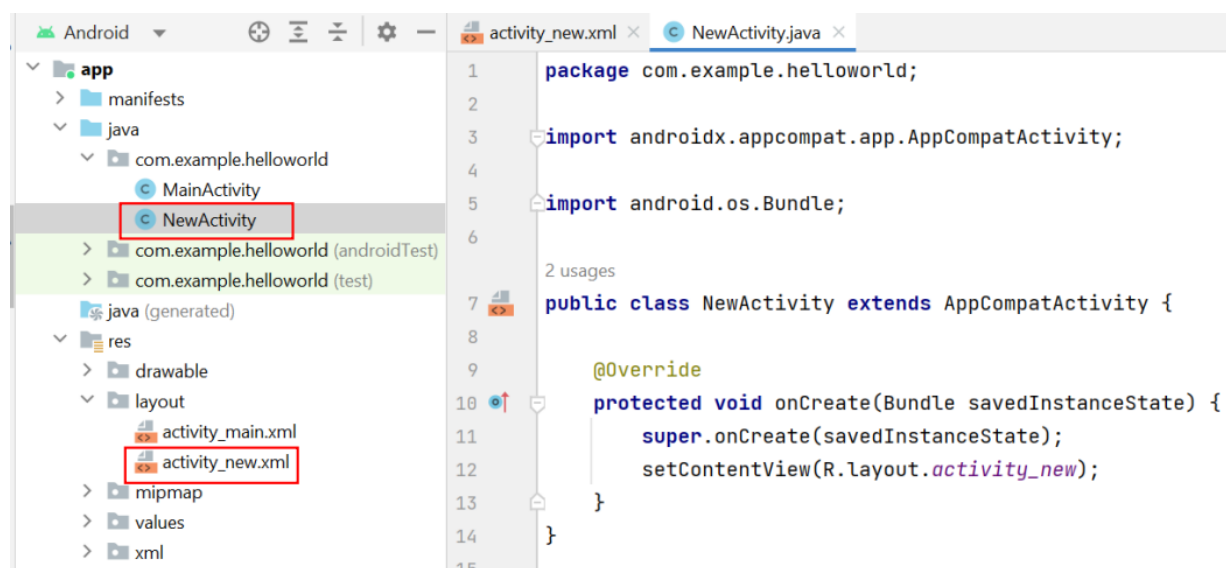


Рисунок 1.4.3 — Сгенерированные файлы, связанные с новой activity

Переход между разными экранами в приложениях Android осуществляется при помощи механизма, называемого "Intent"(или намерение). Это основная концепция в Android, которая играет важную роль в обеспечении взаимодействия между различными компонентами приложения, в том числе активностями, которые представляют собой отдельные экраны в приложении.

Intent можно описать как сообщение или запрос, который указывает на намерение выполнить определенное действие. В контексте перехода между экранами, Intent используется для запуска новой активности. Он не только указывает системе, что нужно перейти на другой экран, но и может переносить информацию от одного экрана к другому. Например, если приложение содержит список элементов и детальный просмотр каждого элемента на отдельном экране, Intent может использоваться для запуска активности детального просмотра и передачи данных о выбранном элементе.

Создание Intent включает в себя указание контекста (например, текущей активности) и класса активности, которую необходимо запустить. Пример создания Intent для запуска новой активности представлен на рисунке 1.4.4.

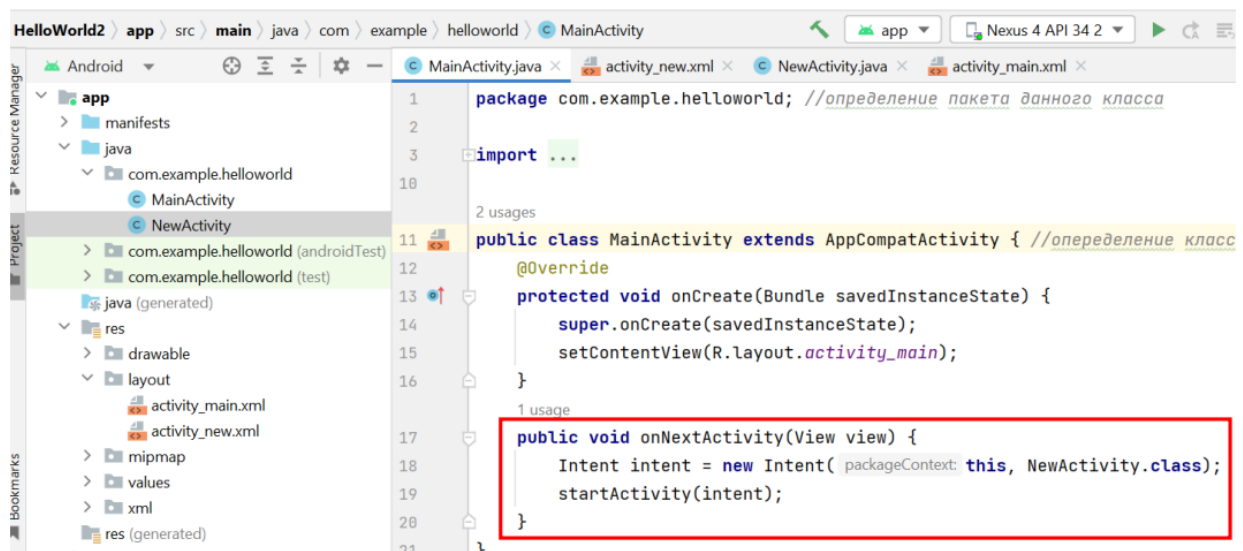


Рисунок 1.4.4 — Пример создания Intent для запуска новой активности

Здесь "this" обозначает текущую активность, а "NewActivity" — созданную ранее активность, которую нужно запустить, далее метод "startActivity", начинает выполнение действия, определенного в объекте Intent. По выполнении этой строки, система Android обрабатывает Intent и запускает активность, указанную в нем. Это приводит к переключению пользовательского интерфейса с текущей активности на интерфейс новой активности "NewActivity" (рисунок 1.4.5).

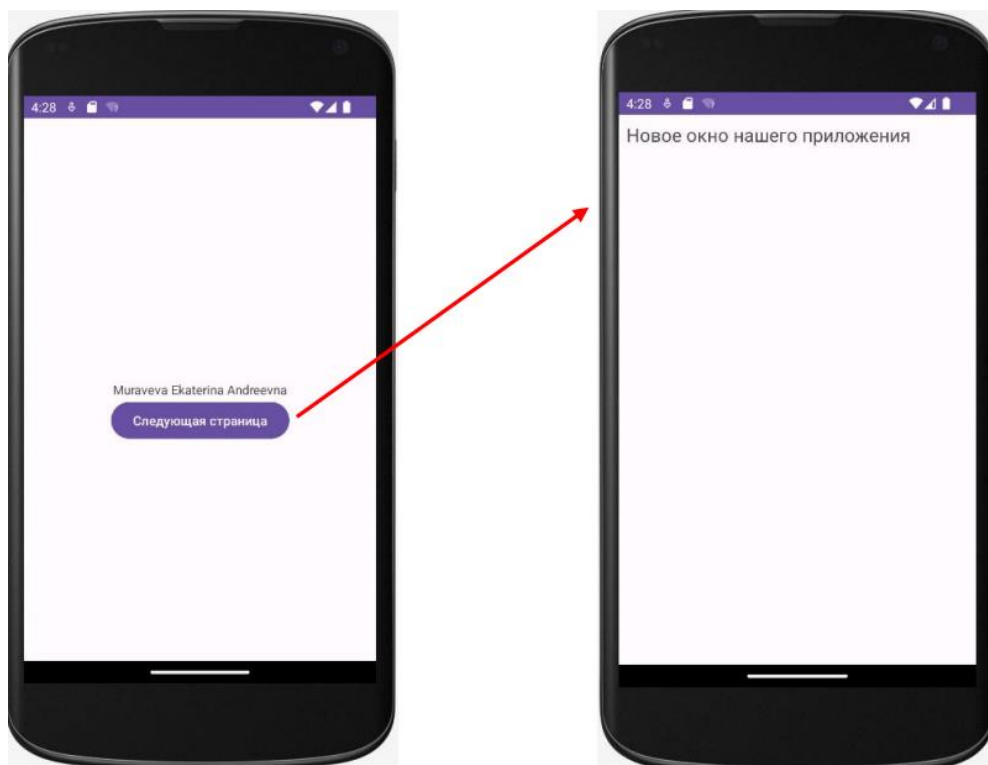


Рисунок 1.4.5 — Переход от одной activity к другой

Таким образом, использование нескольких активностей и механизма Intent позволяет создавать многоуровневые и взаимосвязанные пользовательские интерфейсы в Android-приложениях.

1.5 Передача данных между Activity

Простой запуск Activity может быть недостаточен для разработки целых приложений и в определенный момент может понадобиться передать некоторые данные как в открываемое Activity, так и обратно.

В этом случае нужно задействовать специальную область `extraData`, который имеется у класса `Intent`.

Область `extraData` – это список пар ключ/значение, который передаётся вместе с намерением. В качестве ключей используются строки, а для значений можно использовать любые примитивные типы данных: `String`, `int`, `float`, `double`, `long`, `short`, `byte`, `char`; массивы примитивов, объекты класса `Bundle` и др.

Для передачи данных в другую активность используется метод `putExtra()` (рисунок 1.5.1).

```
intent.putExtra("Ключ", "Значение");
```

Рисунок 1.5.1 — Передача данных в другую активность

Чтобы получить отправленные данные при загрузке `NewActivity`, можно воспользоваться методом `get()`, в который передается ключ объекта (рисунок 1.5.2).

```
Bundle arguments = getIntent().getExtras();  
String name = arguments.get("Ключ").toString();
```

Рисунок 1.5.2 — Получение данных в новой активности

В зависимости от типа отправляемых данных при их получении мы можем использовать ряд методов объекта `Bundle`. Все они в качестве параметра принимают ключ объекта.

Пусть у нас имеется разметка для ввода двух полей: имени и возраста. Создадим поля и присвоим им идентификаторы `name` и `age` соответственно. После этого в файле `MainActivity` с помощью метода `putExtra` передаем данные в другую `Activity` (рисунок 1.5.3).

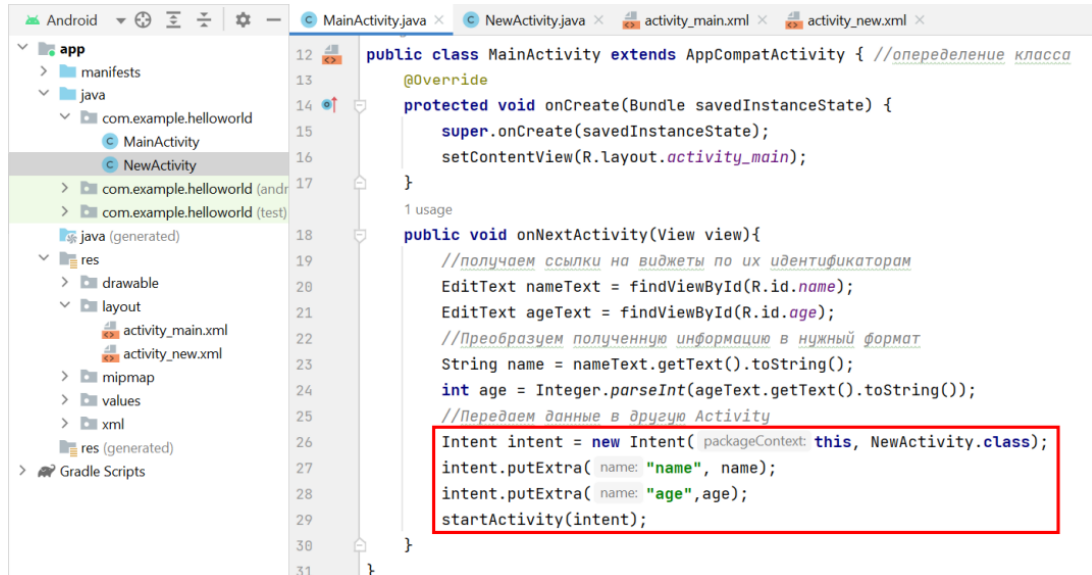


Рисунок 1.5.3 — Передача данных из `MainActivity`

А в другой `NewActivity` наоборот получаем отправленные данные с помощью метода `get()`. Также проверяем, что аргументы имеют хотя бы один символ и не являются пустыми (рисунок 1.5.4).

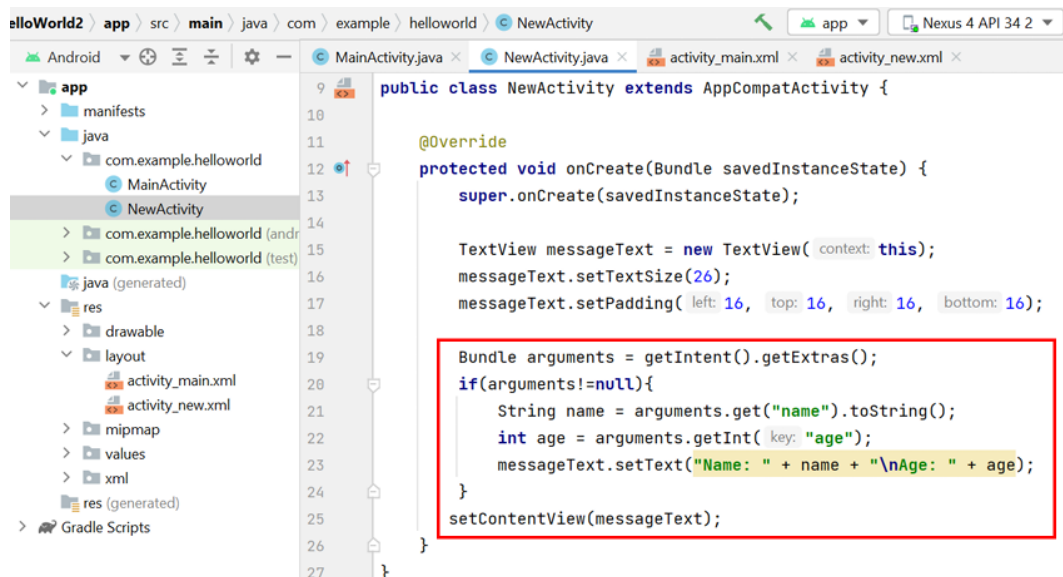


Рисунок 1.5.4 — Получение данных в `NewActivity`

В результате передаем введенные данные из одного окна в другое (рисунок 1.5.5).

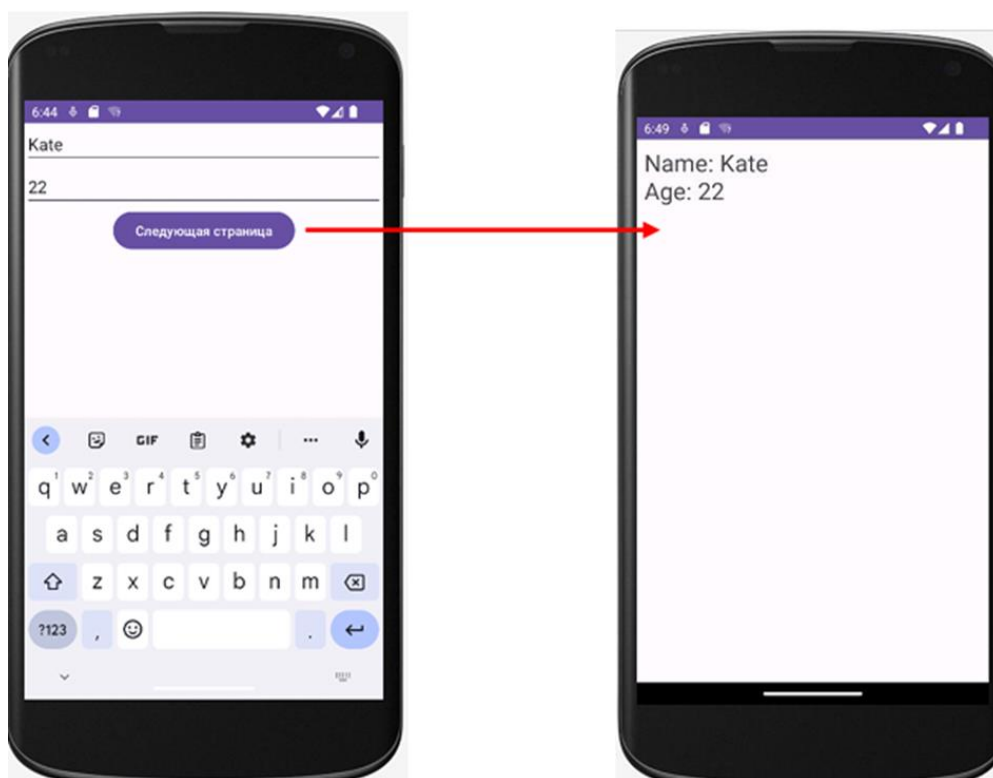


Рисунок 1.5.5 — Результат передачи данных из одного окна в другое

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Отслеживание жизненного цикла активности при помощи логирования

2.1.1 Реализация логирования в методах жизненного цикла

Для создания проекта был выбран шаблон "Empty Views Activity", поскольку его более чем достаточно для решения поставленной задачи. (рисунок 2.1.1.1).

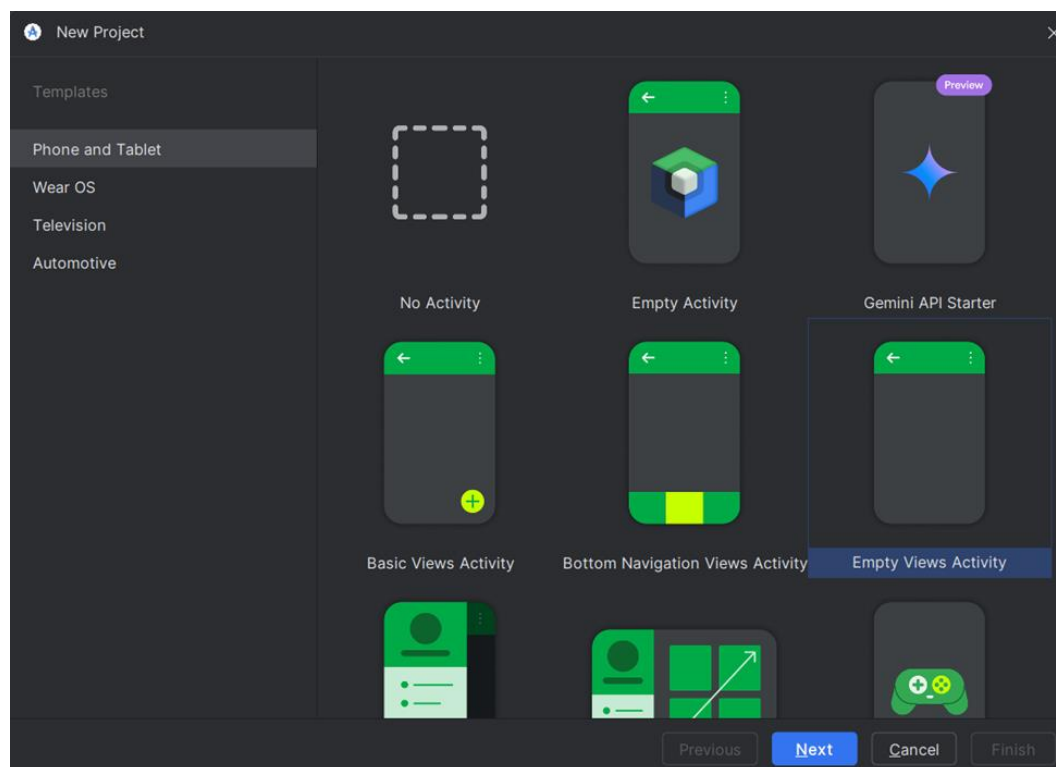


Рисунок 2.1.1.1 — Выбор шаблона проекта в Android Studio

MainActivity изначально состоит из метода onCreate(), который устанавливает разметку activity_main, включает поддержку EdgeToEdge и настраивает обработку системных отступов через ViewCompat.setOnApplyWindowInsetsListener() (рисунок 2.1.1.2).

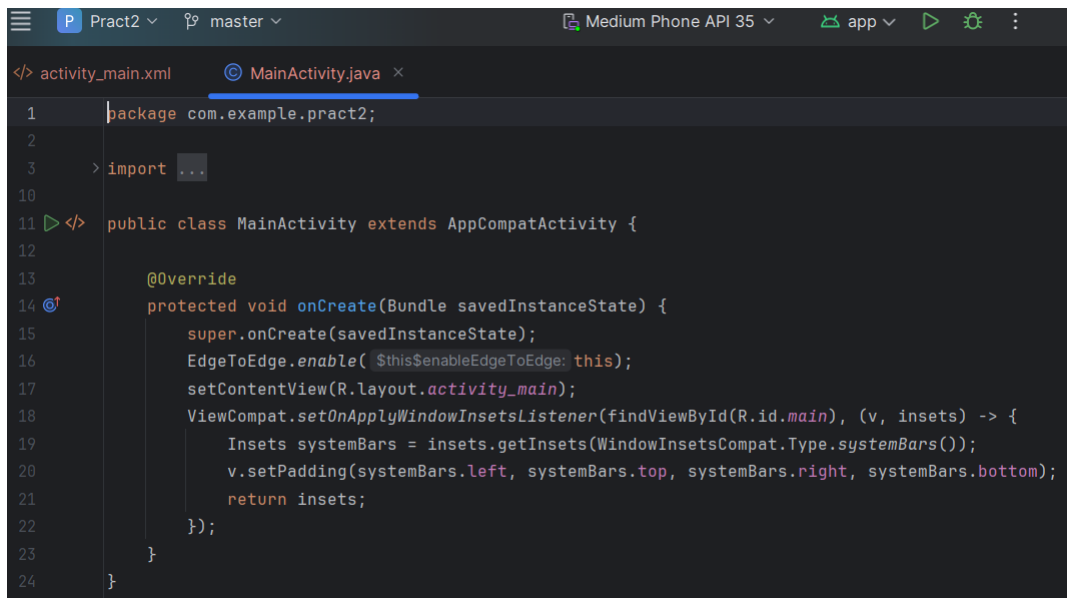


Рисунок 2.1.1.2 — Изначальный вид MainActivity

Определим в поле класса MainActivity пользовательский тег для идентификации сообщений логирования (рисунок 2.1.1.3).

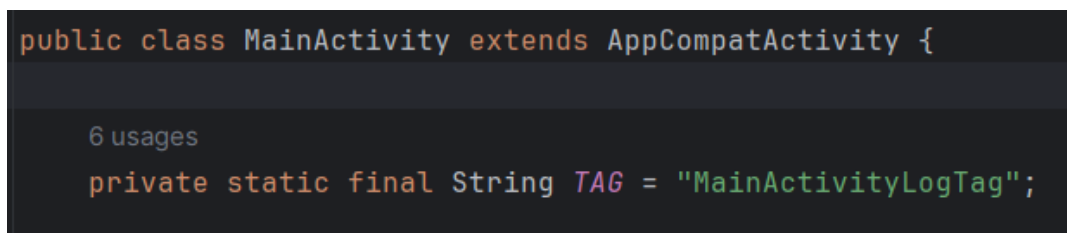


Рисунок 2.1.1.3 — Пользовательский тег для сообщений логирования

Добавим в конец метода onCreate() запись лога с помощью вызова метода Log.i(), где в качестве тега используется ранее объявленное в поле TAG класса MainActivity значение пользовательского тега, а в качестве сообщения используется название метода, то есть "onCreate" (рисунок 2.1.1.4).



Рисунок 2.1.1.4 — Сообщение логирования в методе onCreate()

Далее необходимо добавить сообщение логирования в методы onCreate(), onStart(), onResume(), onPause(), onStop(), onDestroy(). Воспользуемся возможностями Android Studio и откроем окно выбора методов для определения с помощью сочетания клавиш "Ctrl+O", где с помощью поиска, найдём нужные нам методы (рисунок 2.1.1.5).

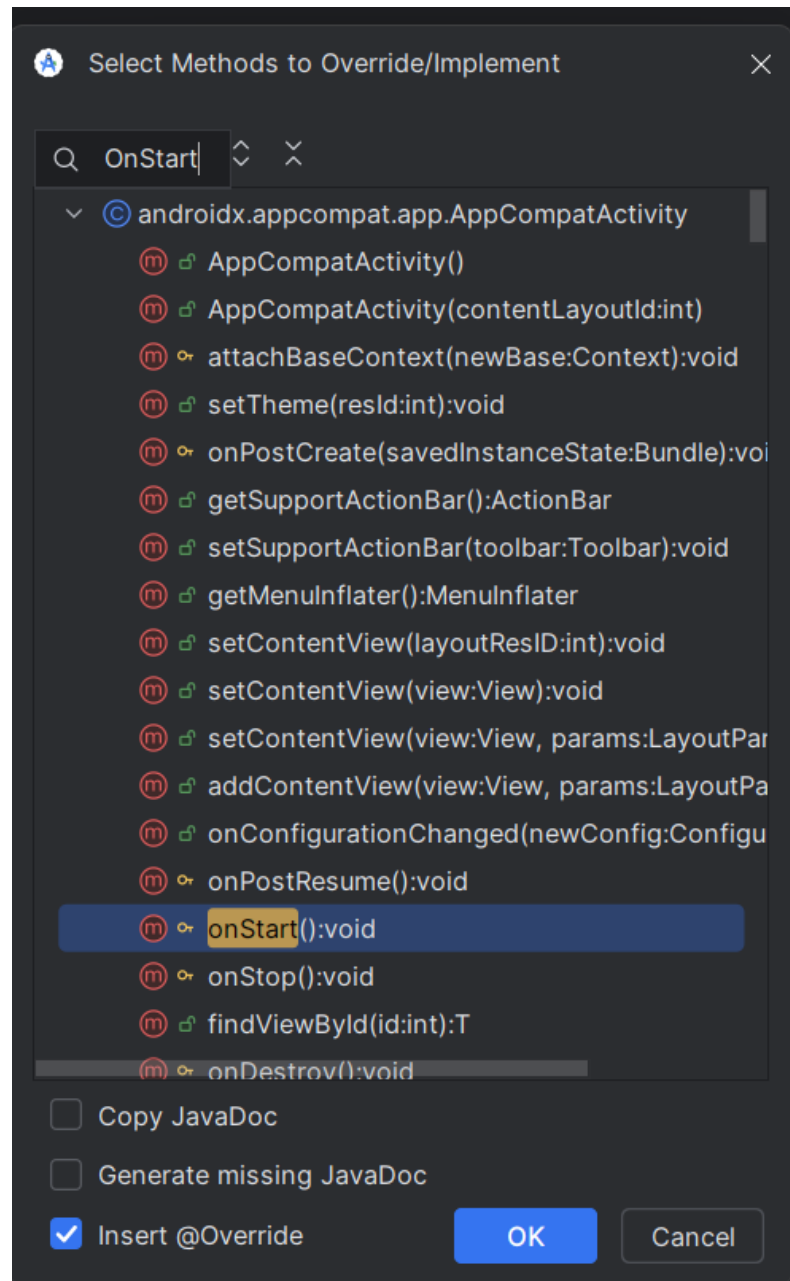


Рисунок 2.1.1.5 — Окно переопределения методов

Android Studio автоматически создаст переопределённый метод, в теле которого находится лишь вызов переопределяемого метода с помощью ключевого слова super (рисунок 2.1.1.6).

```

@Override
protected void onStart() {
    super.onStart();
}

```

Рисунок 2.1.1.6 — Переопределённый средой разработки Android Studio метод

Добавим в конец тела каждого из таких методов вывод записей лога по аналогии с добавлением записи лога в метод onCreate() (рисунок 2.1.1.7).

```

@Override
protected void onStart() {
    super.onStart();
    Log.i(TAG, msg: "onStart");
}

@Override
protected void onResume() {
    super.onResume();
    Log.i(TAG, msg: "onResume");
}

@Override
protected void onPause() {
    super.onPause();
    Log.i(TAG, msg: "onPause");
}

@Override
protected void onStop() {
    super.onStop();
    Log.i(TAG, msg: "onStop");
}

@Override
protected void onDestroy() {
    super.onDestroy();
    Log.i(TAG, msg: "onDestroy");
}

```

Рисунок 2.1.1.7 — Логирование в переопределённых методах

После добавления методов для записи в лог в каждый из нужных методов. Откроем панель Logcat, расположенную в левом нижнем углу

(рисунок 2.1.1.8). В ней выберем созданный ранее пользовательский тег в качестве фильтра записей (рисунок 2.1.1.9).

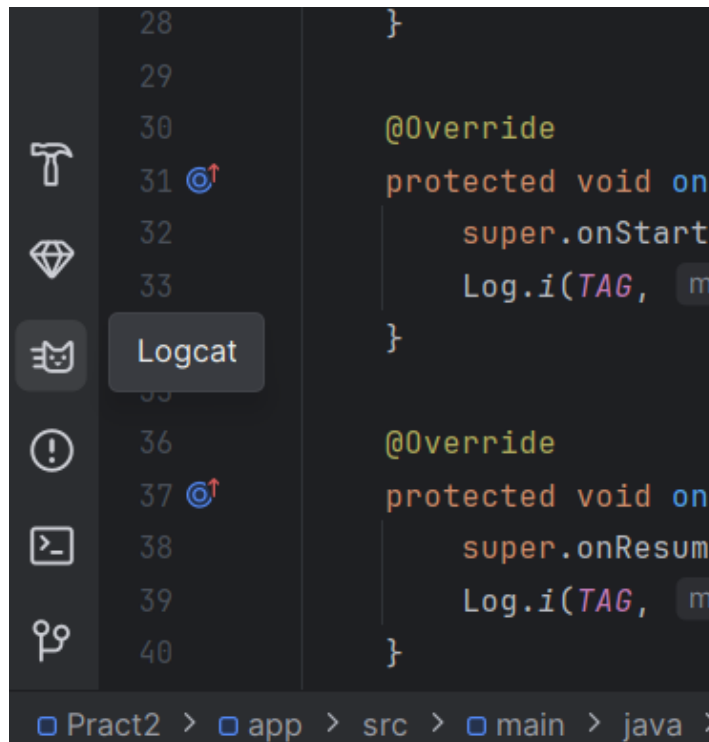


Рисунок 2.1.1.8 — Открытие панели Logcat

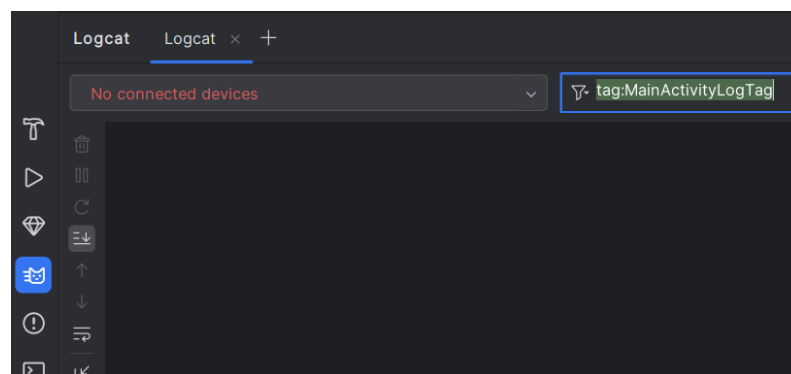


Рисунок 2.1.1.9 — Выбор созданного ранее пользовательского тега в качестве фильтра записей

2.1.2 Тестирование отслеживания жизненного цикла активности

Протестируем отслеживание жизненного цикла активности. Запустим наше приложение и увидим, что в панели Logcat отобразились сообщения из методов `onCreate()`, `onStart()` и `onResume()`, поскольку `MainActivity` была создана, стала видимой и приняла готовность к взаимодействию с пользователем (рисунок 2.1.2.1).

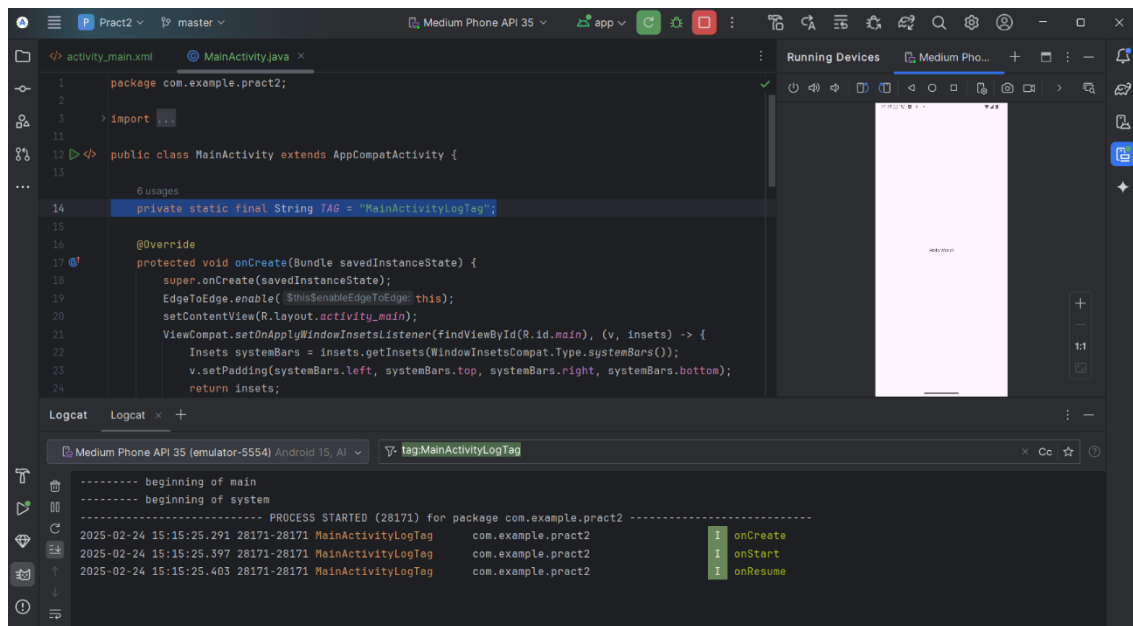


Рисунок 2.1.2.1 — Тестирование логирования жизненного цикла активности (часть 1)

Перейдём на домашний экран Android и увидим, что в панели Logcat отобразились сообщения из методов onPause() и onStop(), так как MainActivity сначала перестала быть доступной для взаимодействия с пользователем, а затем перестала быть видимой (рисунок 2.1.2.2).

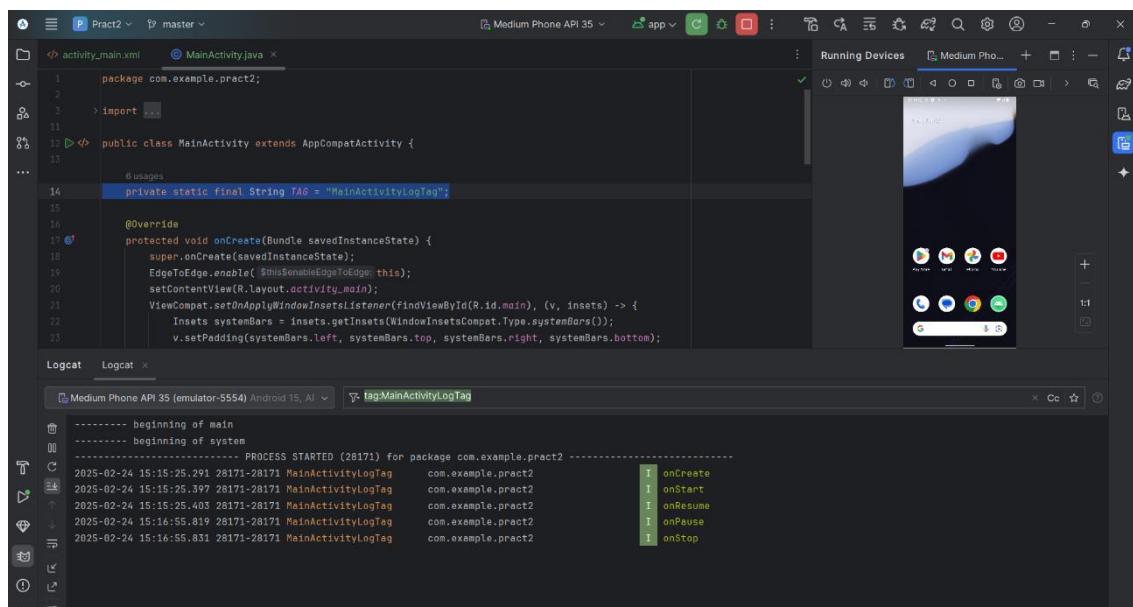


Рисунок 2.1.2.2 — Тестирование логирования жизненного цикла активности (часть 2)

Вернёмся к нашему приложению и увидим, что вновь были вызваны методы onStart() и onResume(), поскольку MainActivity вновь стала видимой и готовой к взаимодействию с пользователем (рисунок 2.1.2.3).

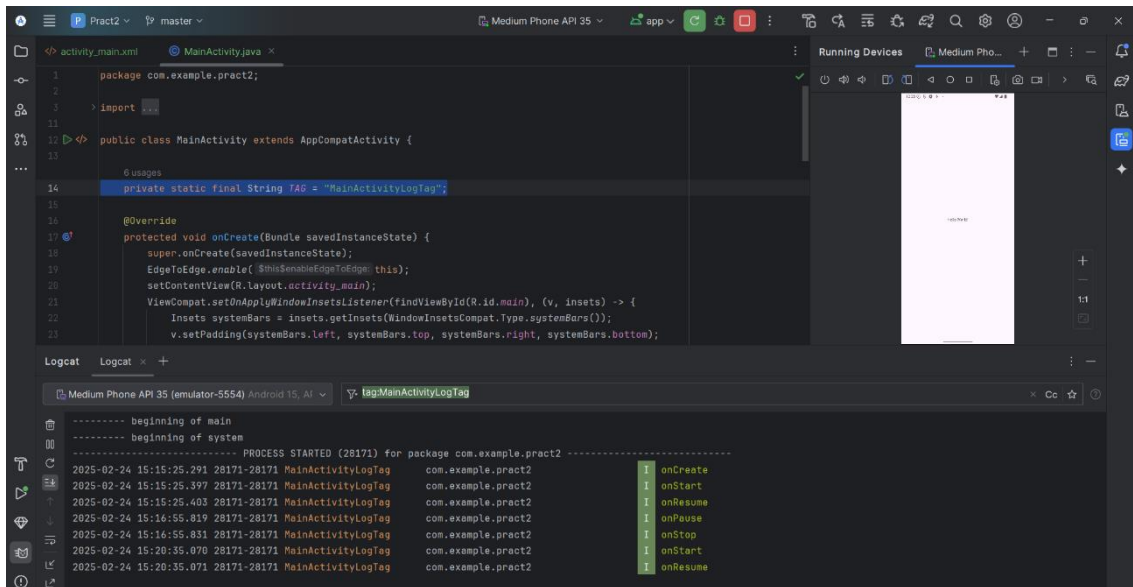


Рисунок 2.1.2.3 — Тестирование логирования жизненного цикла активности (часть 3)

Завершив работу с приложением, закрыв его полностью, и увидим записи логов из методов onPause(), onStop() и onDestroy(), так как MainActivity перестала быть доступным для взаимодействия с пользователем, перестала быть видимой и завершила свою работу (рисунок 2.1.2.4). Можно заключить, что логирование работы жизненного цикла активности работает корректно.

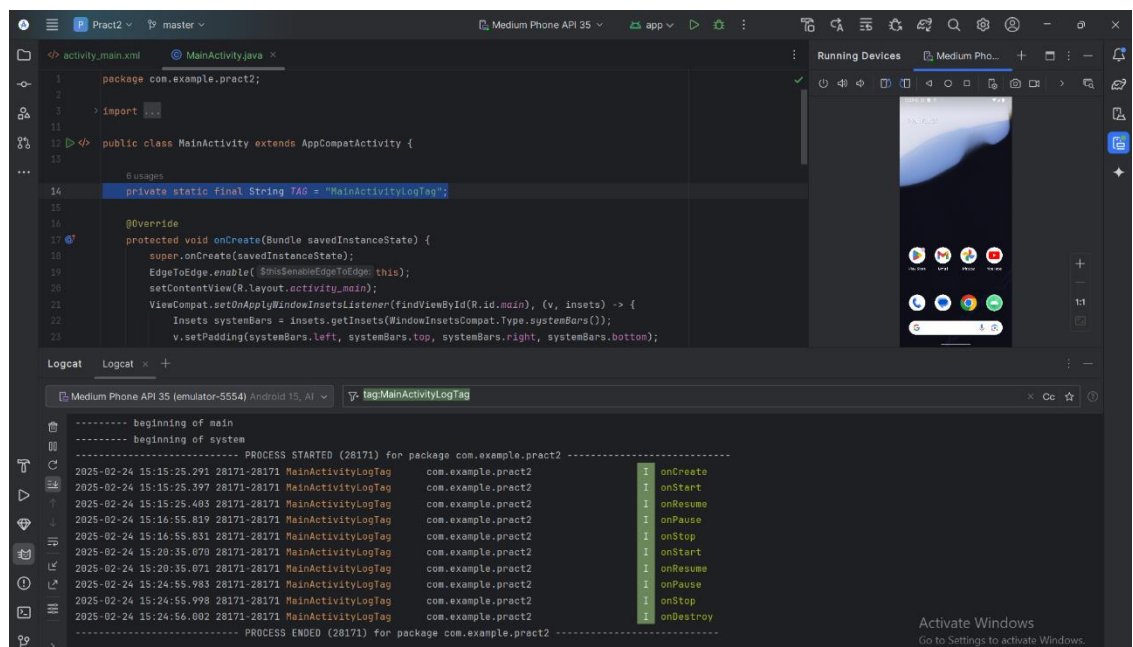
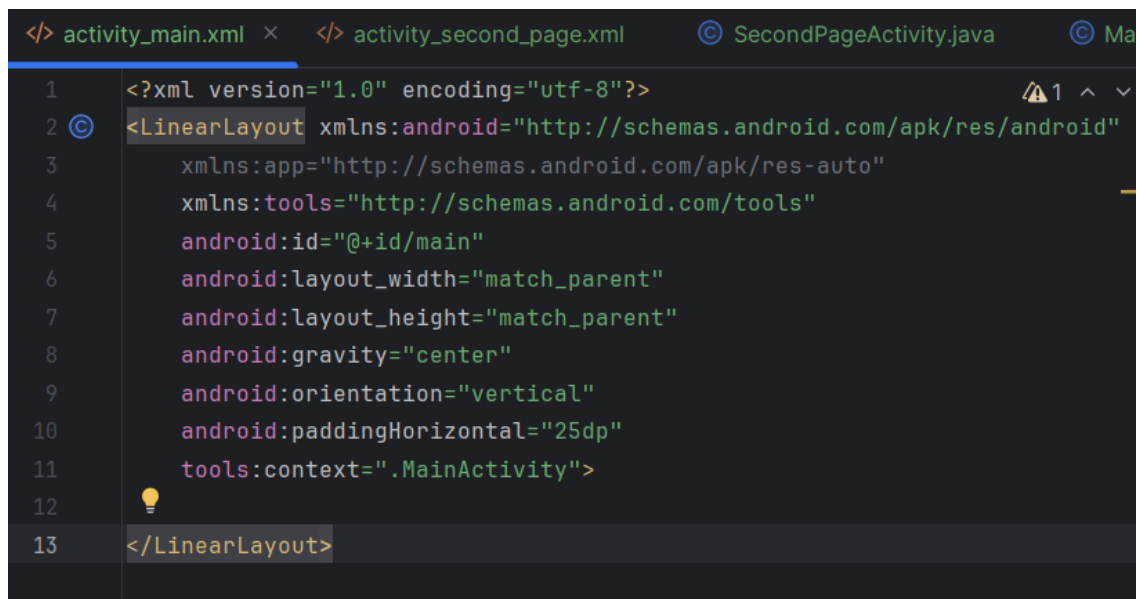


Рисунок 2.1.2.4 — Тестирование логирования жизненного цикла активности (часть 4)

2.2 Переход на другую активность

2.2.1 Создание разметки первой активности

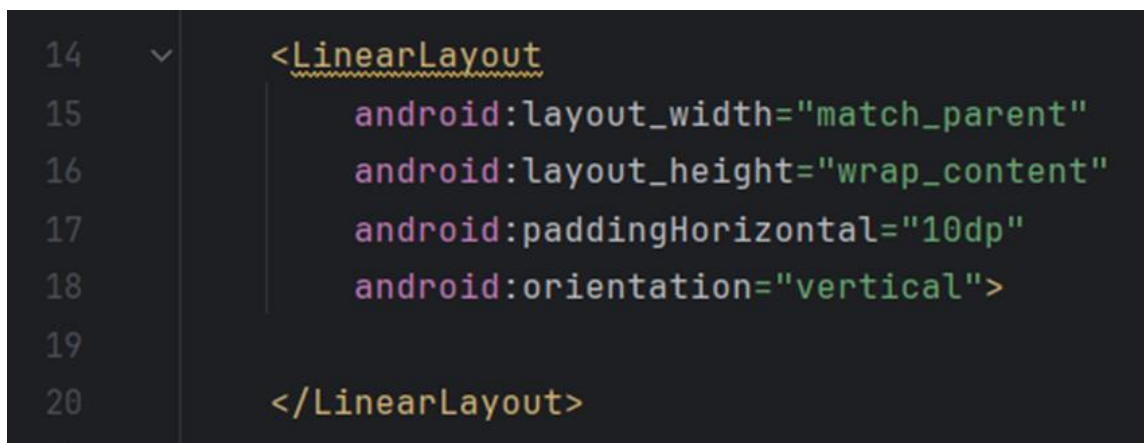
В файле `activity_main.xml` корневым элементом установим `Linear Layout`, с вертикальной ориентацией, выравниванием элементов по центру как горизонтальной, так и вертикальной оси, и горизонтальными внутренними отступами (рисунок 2.2.1.1).



```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:paddingHorizontal="25dp"
    tools:context=".MainActivity">
</LinearLayout>
```

Рисунок 2.2.1.1 — Создание корневого элемента разметки первой активности

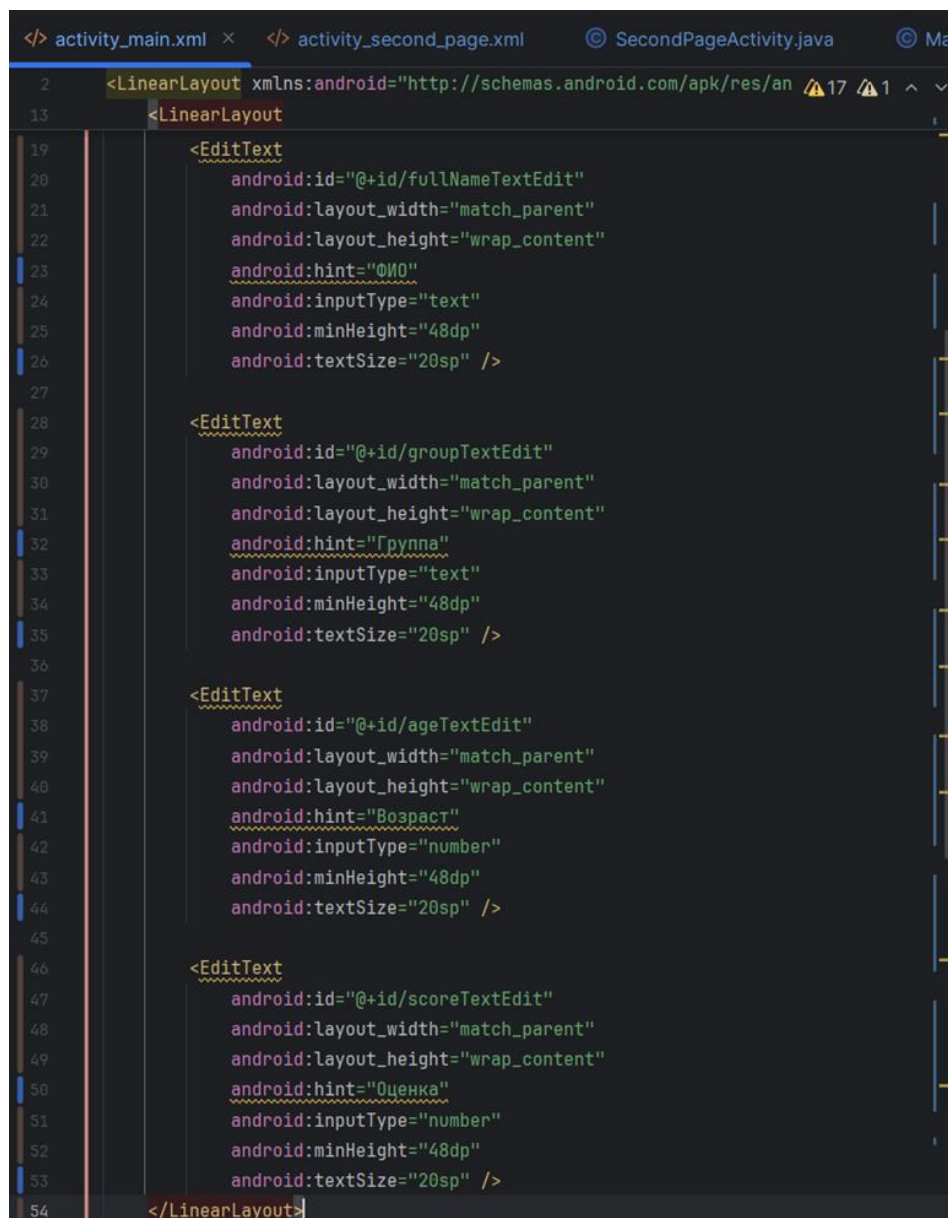
Далее добавим вложенный `LinearLayout`, в котором будут располагаться поля для ввода, с вертикальной ориентацией, и горизонтальными внутренними отступами, шириной равной ширине родительского компонента и высотой в зависимости от содержимого (рисунок 2.2.1.2).



```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:paddingHorizontal="10dp"
    android:orientation="vertical">
</LinearLayout>
```

Рисунок 2.2.1.2 — Вложенный `LinearLayout`

В качестве дочерних элементов рассмотренного выше `LinearLayout` добавим четыре поля для ввода типа `EditText` (рисунок 2.2.1.3). Каждое из них будет занимать всю ширину родителя, иметь высоту в зависимости от содержания. Значения текстов-подсказок: «ФИО», «Группа», «Возраст» и «Оценка» соответственно. Каждому компоненту присвоим осмысленное `id`, которое в дальнейшем будет делать код читаемым.



```
<?xml version="1.0" encoding="utf-8" android:namespace="http://schemas.android.com/apk/res/android"
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="com.example.myapplication.MainActivity"
    tools:show="assistant">
    <LinearLayout
        android:id="@+id/fullNameTextEdit"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="ФИО"
        android:inputType="text"
        android:minHeight="48dp"
        android:textSize="20sp" />
    <EditText
        android:id="@+id/groupTextEdit"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Группа"
        android:inputType="text"
        android:minHeight="48dp"
        android:textSize="20sp" />
    <EditText
        android:id="@+id/ageTextEdit"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Возраст"
        android:inputType="number"
        android:minHeight="48dp"
        android:textSize="20sp" />
    <EditText
        android:id="@+id/scoreTextEdit"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Оценка"
        android:inputType="number"
        android:minHeight="48dp"
        android:textSize="20sp" />
</LinearLayout>
```

Рисунок 2.2.1.3 — Поля для ввода

Затем под группой полей для ввода добавим два компонента-кнопки разных цветов (рисунок 2.2.1.4). Посмотрим на получившийся файл разметки в графическом режиме (рисунок 2.2.1.5).

```

</LinearLayout>

<Button
    android:id="@+id/declarativeButton"
    android:layout_width="206dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="30dp"
    android:text="Подтвердить"
    android:textSize="20sp" />

<Button
    android:id="@+id/programmedButton"
    android:layout_width="206dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:backgroundTint="#E91E63"
    android:text="Подтвердить"
    android:textColorLink="#AF2121"
    android:textSize="20sp" />

</LinearLayout>

```

Рисунок 2.2.1.4 — Две кнопки

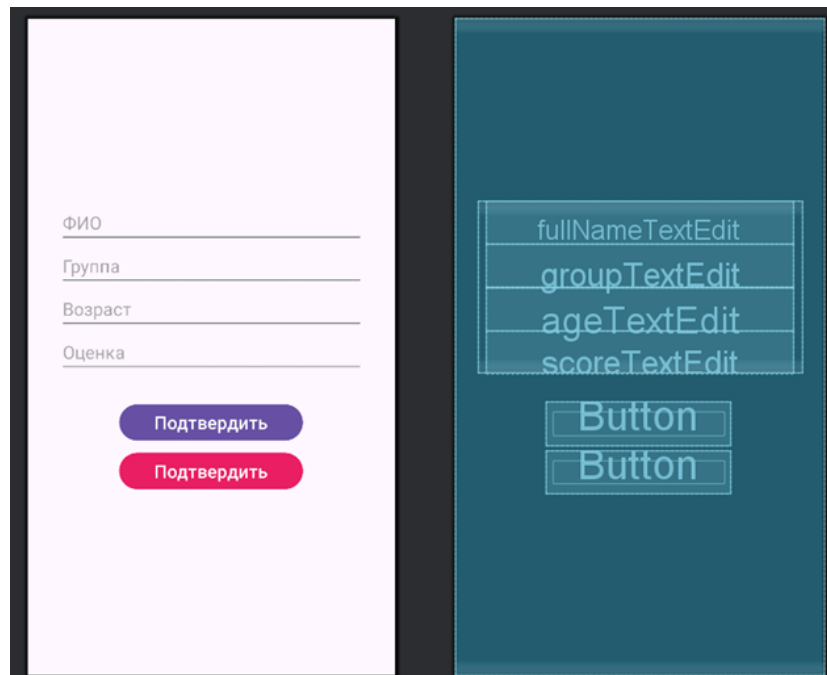


Рисунок 2.2.1.5 — Разметка MainActivity

2.2.2 Создание второй активности и её разметки

Создадим вторую активность и присвоим ей название `SecondPageActivity` (рисунки 2.2.2.1–2.2.2.2).

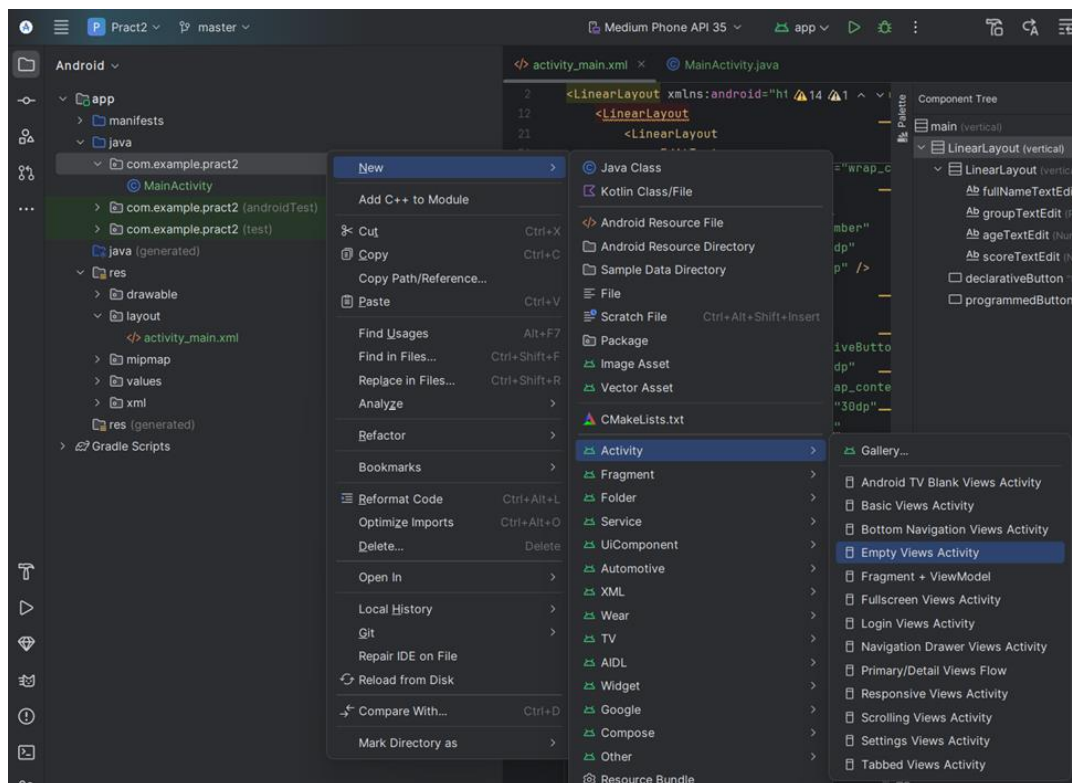


Рисунок 2.2.2.1 — Создание второй активности с помощью выпадающих меню

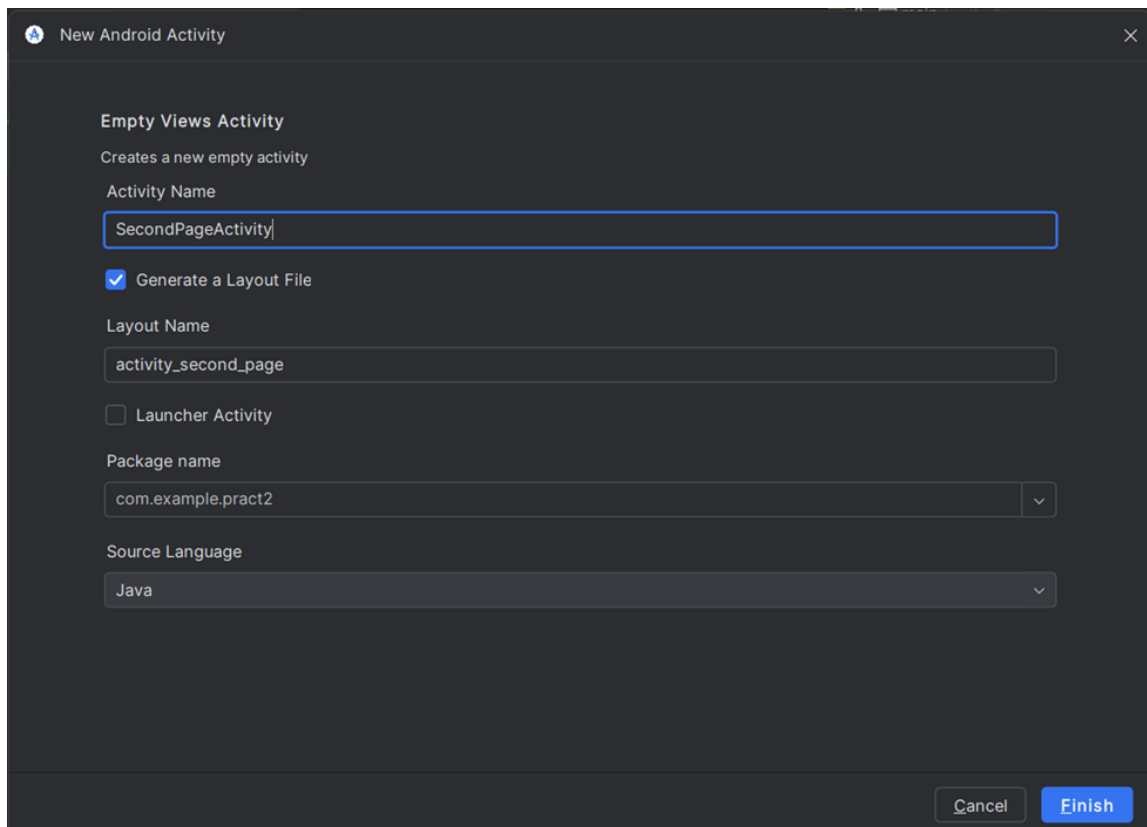


Рисунок 2.2.2.2 — Настройка создания второй активности

В файле `activity_second_page.xml` корневым элементом установим `Linear Layout`, с вертикальной ориентацией, выравниванием элементов по центру как горизонтальной, так и вертикальной оси, и горизонтальными внутренними отступами (рисунок 2.2.2.3).

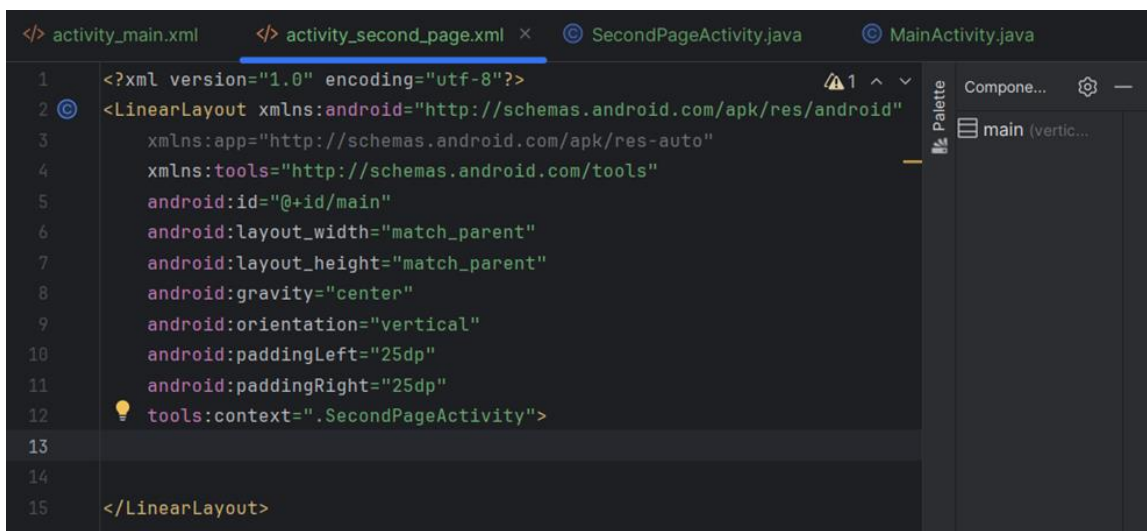


Рисунок 2.2.2.3 — Создание корневого элемента разметки первой активности

Далее добавим четыре компонента `TextView` для отображения текста (рисунок 2.2.2.4). Присвоим им осмысленные `id` и текстовые значения:

«ФИО», «Группа», «Возраст» и «Оценка» соответственно. Текст этих элементов будет выставлен программно при переходе в данную активность из MainActivity, однако на данном этапе всё равно полезно присвоить значения атрибутам android:text, чтобы можно было визуально оценить разметку до запуска приложения (рисунок 2.2.2.5).

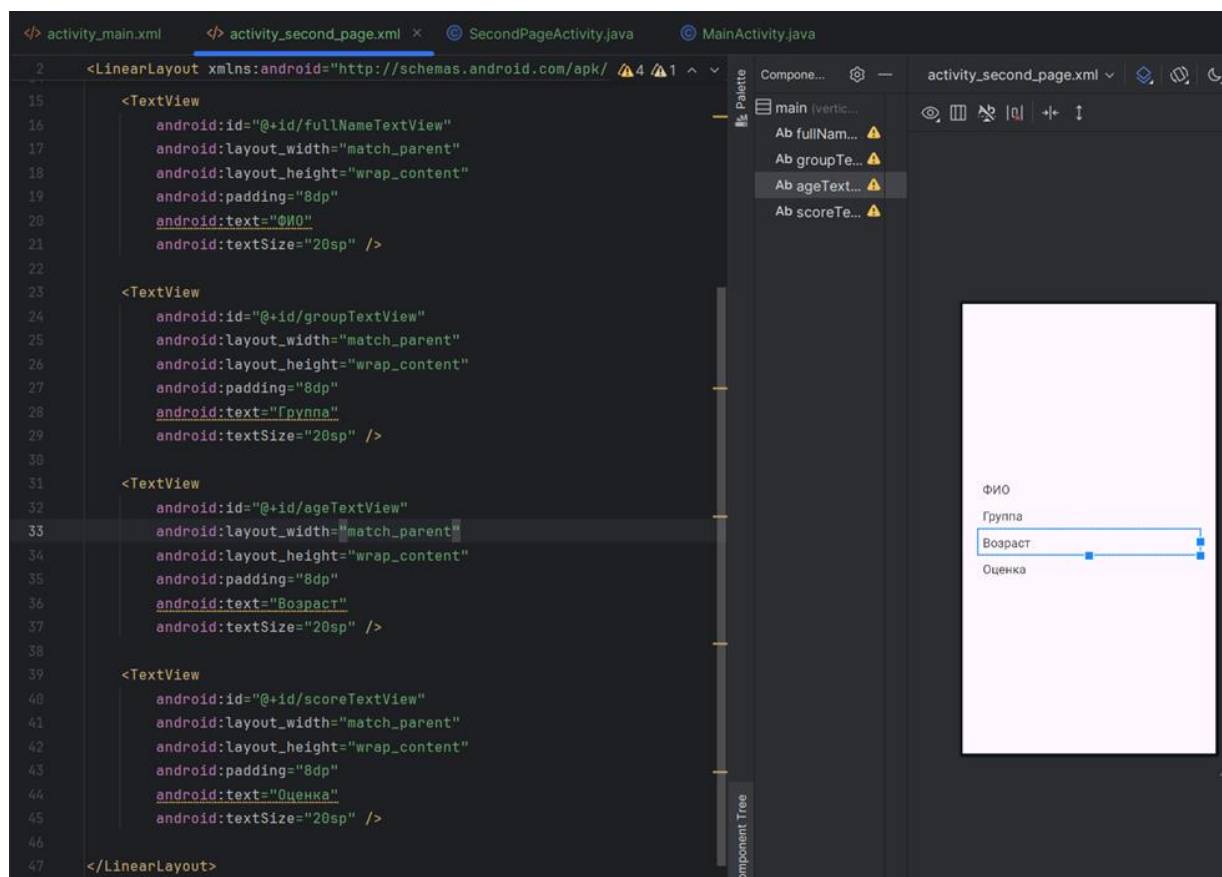


Рисунок 2.2.2.5 — Разметка второй активности

2.2.3 Реализация перехода на другую активность

В классе MainActivity напомним метод обработчик событий navigateToSecondPage(). В нём создаётся, а затем передаётся методу startActivity(), объект класса Intent, который используется для указания системе, что нужно запустить новую активность. В нашем случае из текущей активности MainActivity запускается активность SecondPageActivity (рисунок 2.2.3.1).

```

15 public class MainActivity extends AppCompatActivity {
33     public void navigateToSecondPage(View view) {
34         Intent intent = new Intent( packageContext: this, SecondPageActivity.class);
35         startActivity(intent);
36     }

```

Рисунок 2.2.3.1 — Создание обработчика событий для перехода на другую активность

Затем в методе `onCreate()` класса `MainActivity` получаем ссылку на компонент кнопки с `id programmedButton` и программно устанавливаем метод `navigateToSecondPage()` в качестве его обработчика событий с помощью метода `setOnClickListener()` в Java коде активности (рисунок 2.2.3.2).

```

16 public class MainActivity extends AppCompatActivity {
21     protected void onCreate(Bundle savedInstanceState) {
31         Button programmedButton = findViewById(R.id.programmedButton);
32         programmedButton.setOnClickListener(this::navigateToSecondPage);
33
34         Log.i(TAG, msg: "onCreate");
35     }
36
37     1 usage
38     public void navigateToSecondPage(View view) {

```

Рисунок 2.2.3.2 — Установка обработчика событий для кнопки программно

Для кнопки с `id declarativeButton` устанавливаем тот же метод `navigateToSecondPage()` в качестве обработчика событий декларативным способом в файле разметки `activity_main.xml` с помощью атрибута `onClick` (рисунок 2.2.3.3).

```

2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
14     <LinearLayout
49         <EditText
52             android:layout_height="wrap_content"
53             android:ems="10"
54             android:hint="Оценка"
55             android:inputType="number"
56             android:minHeight="48dp"
57             android:textSize="20sp" />
58     </LinearLayout>
59
60     <Button
61         android:id="@+id/declarativeButton"
62         android:onClick="navigateToSecondPage"
63         android:layout_width="206dp"
64         android:layout_height="wrap_content"
65         android:layout_marginTop="30dp"
66         android:text="Подтвердить"
67         android:textSize="20sp" />

```

Рисунок 2.2.3.3 — Установка обработчика событий для кнопки

2.2.4 Реализация передачи данных из первой активности во вторую

В начале тела обработчика событий `navigateToSecondPage()` получим ссылки на каждое из полей ввода `MainActivity`, а затем извлечём из них текстовые значения и присвоим их строковым переменным с помощью методов `getText()` и `toString()` (рисунок 2.2.4.1).

```
public void navigateToSecondPage(View view) {  
    EditText fullNameEditText = findViewById(R.id.fullNameTextEdit);  
    EditText groupEditText = findViewById(R.id.groupTextEdit);  
    EditText ageEditText = findViewById(R.id.ageTextEdit);  
    EditText scoreEditText = findViewById(R.id.scoreTextEdit);  
  
    String fullName = fullNameEditText.getText().toString();  
    String group = groupEditText.getText().toString();  
    String age = ageEditText.getText().toString();  
    String score = scoreEditText.getText().toString();|
```

Рисунок 2.2.4.1 — Извлечение значений из полей ввода

Далее с помощью метода `putExtra()` добавим в объект класса `Intent` значения передаваемых данных (рисунок 2.2.4.2).

```
16 public class MainActivity extends AppCompatActivity {  
36  
    2 usages  
37 public void navigateToSecondPage(View view) {  
38     EditText fullNameEditText = findViewById(R.id.fullNameTextEdit);  
39     EditText groupEditText = findViewById(R.id.groupTextEdit);  
40     EditText ageEditText = findViewById(R.id.ageTextEdit);  
41     EditText scoreEditText = findViewById(R.id.scoreTextEdit);  
42  
43     String fullName = fullNameEditText.getText().toString();  
44     String group = groupEditText.getText().toString();  
45     String age = ageEditText.getText().toString();  
46     String score = scoreEditText.getText().toString();  
47  
48     Intent intent = new Intent( packageContext: this, SecondPageActivity.class);  
49     intent.putExtra( name: "fullName", fullName);  
50     intent.putExtra( name: "group", group);  
51     intent.putExtra( name: "age", age);  
52     intent.putExtra( name: "score", score);  
53  
54     startActivity(intent);  
55 }
```

Рисунок 2.2.4.2 — Добавление дополнительных данных в объект класса `Intent`

В методе `onCreate()` класса `SecondPageActivity` выполняется получение переданных данных из `Intent`, который вызвал текущую активность

SecondPageActivity. Метод getIntent() возвращает Intent, который запустил эту активность, а getExtras() извлекает Bundle с дополнительными данными, если они были переданы (рисунок 2.2.4.3).

```
Bundle arguments = getIntent().getExtras();
```

Рисунок 2.2.4.3 — Извлечение из объекта класса Intent объекта класса Bundle

Затем, если объект класса Intent содержал в себе какие-то дополнительные данные, из объекта класса Bundle по ключу извлекаются переданные значения ФИО, группы, возраста и оценки и присваиваются строковым переменным (рисунок 2.2.4.4).

```
Bundle arguments = getIntent().getExtras();
if (arguments != null) {
    String fullName = arguments.getString( key: "fullName");
    String group = arguments.getString( key: "group");
    String age = arguments.getString( key: "age");
    String score = arguments.getString( key: "score");
}
```

Рисунок 2.2.4.4 — Извлечение переданных данных из объекта класса Bundle

Далее по id получаем ссылки на текстовые компоненты активности SecondPageActivity и с помощью метода setText() устанавливаем значение их текста в соответствии с полученными из MainActivity данными (рисунок 2.2.4.5).

```

Bundle arguments = getIntent().getExtras();
if (arguments != null) {
    String fullName = arguments.getString( key: "fullName");
    String group = arguments.getString( key: "group");
    String age = arguments.getString( key: "age");
    String score = arguments.getString( key: "score");

    TextView fullNameTextView = findViewById(R.id.fullNameTextView);
    TextView groupTextView = findViewById(R.id.groupTextView);
    TextView ageTextView = findViewById(R.id.ageTextView);
    TextView scoreTextView = findViewById(R.id.scoreTextView);

    fullNameTextView.setText(String.format("%s: %s", "ФИО", fullName));
    groupTextView.setText(String.format("%s: %s", "Группа", group));
    ageTextView.setText(String.format("%s: %s", "Возраст", age));
    scoreTextView.setText(String.format("%s: %s", "Оценка", score));
}

```

Рисунок 2.2.4.5 — Заполнение текстовых полей переданными значениями

Проведём тестирование созданного приложения. Введём данные в поля ввода первой активности, а затем нажмём на первую кнопку, и убедимся, что в окне новой активности отображаются введённые ранее данные (рисунок 2.2.4.6). Нажатие на вторую кнопку также приводит к желаемому результату.

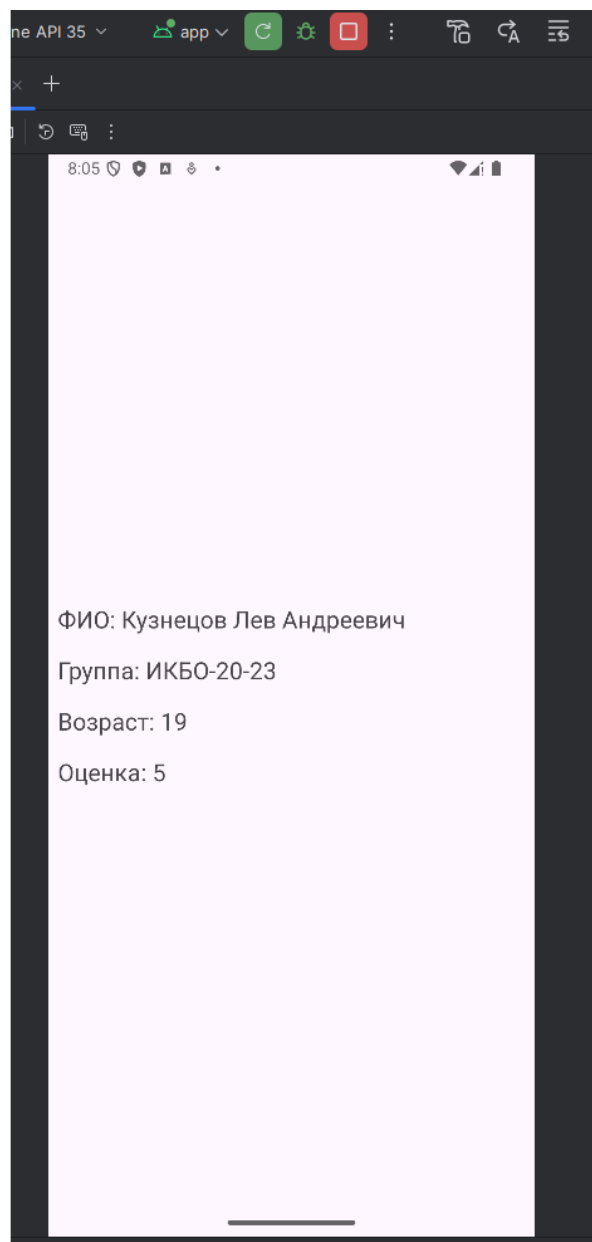
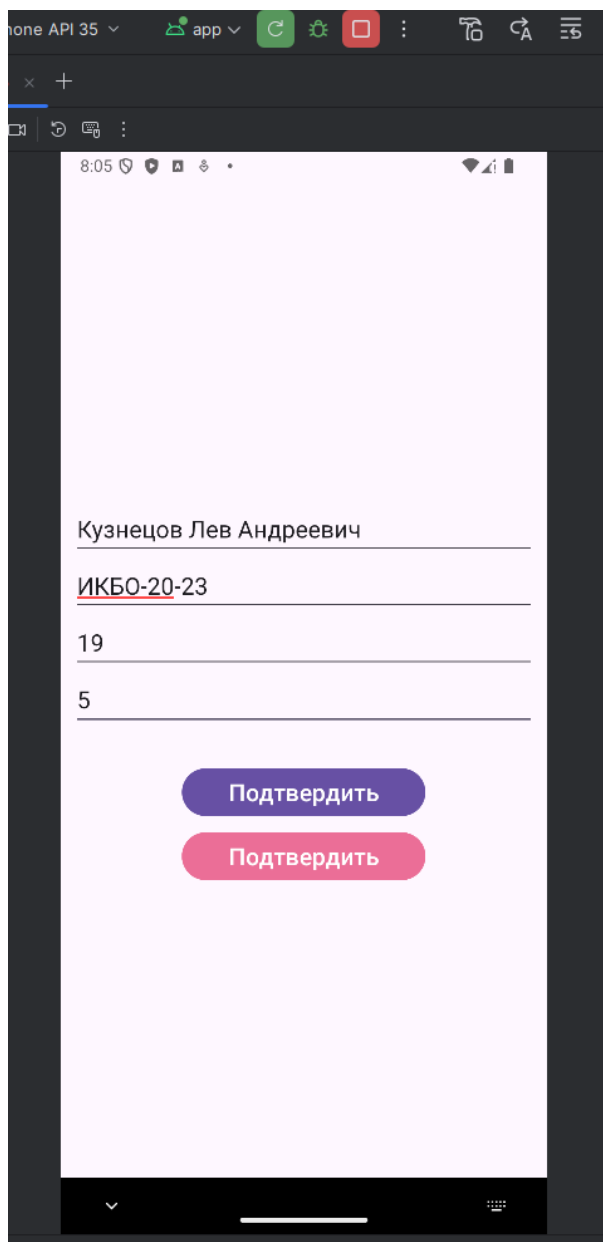


Рисунок 2.2.4.6 — Проверка работы приложения

ЗАКЛЮЧЕНИЕ

По итогу данной работы было проведено знакомство с логированием и просмотром записей логов через Logcat, был отслежен жизненный цикл активности, а также был реализован переход с одной активности на другую с передачей сопутствующих данных.